

FULL STACK DEVELOPMENT

FASE 5 | AULA 06 -

AZURE CONTAINER REGISTRY (ACR)

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS.....	5
O QUE VOCÊ VIU NESTA AULA?	13
REFERÊNCIAS.....	14

EMSE

O QUE VEM POR AÍ?

Boas-vindas, alunos e alunas.

Nesta aula, vocês aprenderão sobre o Azure Container Registry, como usá-lo da forma correta, criar um repositório de imagens e enviar para o Azure a primeira imagem criada localmente, além de subir uma imagem por meio do Azure Pipelines.

Bons estudos!

EMSE

HANDS ON

Nesta aula, realizaremos a criação de uma imagem Docker por meio do Azure Pipelines de forma automatizada, além de enviar para o Azure Container Registry.



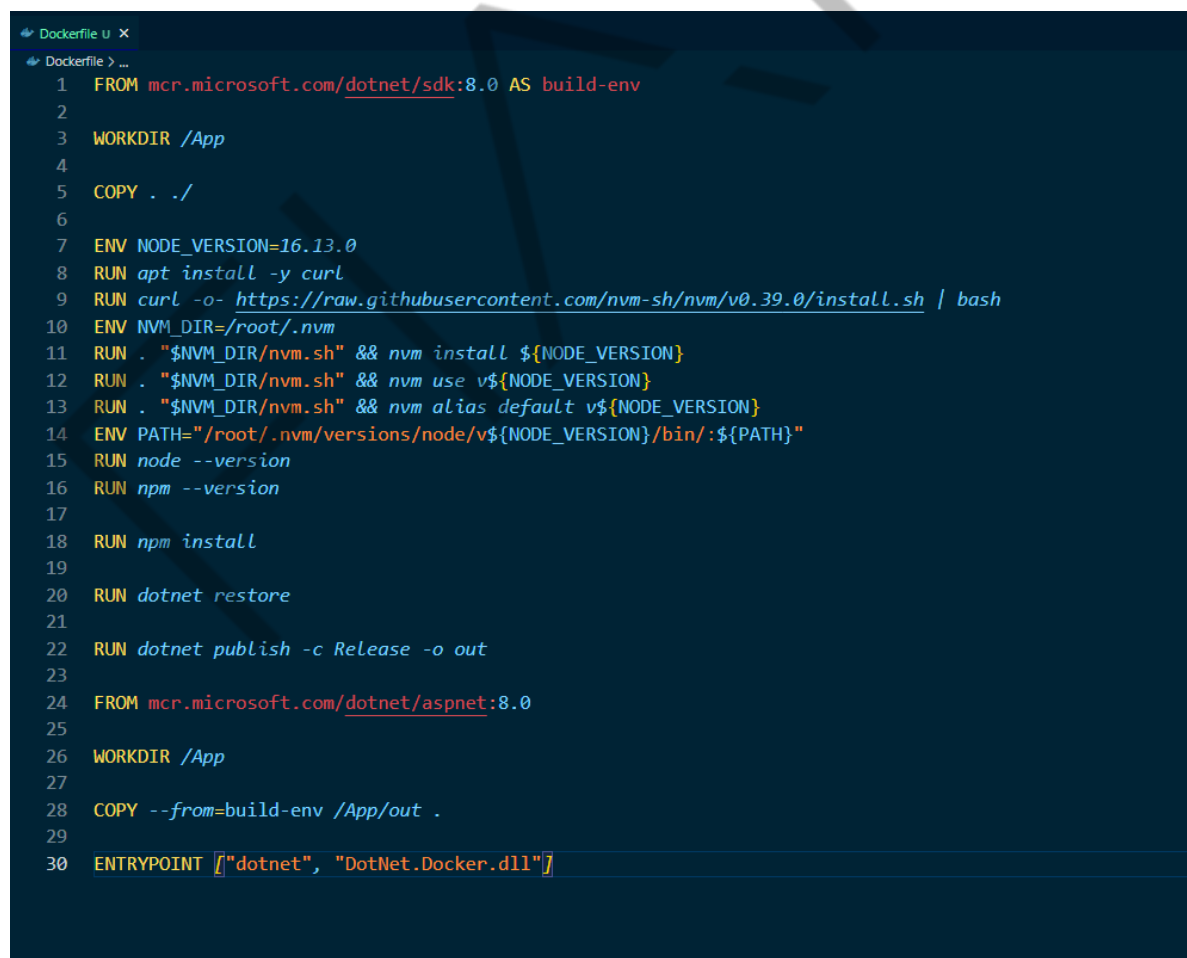
SAIBA MAIS

Nesta aula, faremos o push da imagem a partir do Azure Pipelines e do código hospedado no Azure Repos.

Dessa maneira, todo novo push no Repos iniciará a pipeline em que criará a imagem docker e enviará para o Azure Container Registry de forma automática para que não nos preocupemos com essa entrega. É necessário ter Azure Repos, Azure Pipelines e Azure Container Registry para executar os passos a seguir.

Vamos partir da premissa de que o Repos e o Pipelines já estão criados a partir de aulas anteriores. Então, agora precisamos editar a pipeline adicionando o stage, que faz o build da imagem e em seguida envia para o ACR.

O Dockerfile deve estar parecido com a figura 1.



```
Dockerfile u x
Dockerfile > ...
1 FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build-env
2
3 WORKDIR /App
4
5 COPY . ./
6
7 ENV NODE_VERSION=16.13.0
8 RUN apt install -y curl
9 RUN curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.0/install.sh | bash
10 ENV NVM_DIR=/root/.nvm
11 RUN . "$NVM_DIR/nvm.sh" && nvm install ${NODE_VERSION}
12 RUN . "$NVM_DIR/nvm.sh" && nvm use v${NODE_VERSION}
13 RUN . "$NVM_DIR/nvm.sh" && nvm alias default v${NODE_VERSION}
14 ENV PATH="/root/.nvm/versions/node/v${NODE_VERSION}/bin/:${PATH}"
15 RUN node --version
16 RUN npm --version
17
18 RUN npm install
19
20 RUN dotnet restore
21
22 RUN dotnet publish -c Release -o out
23
24 FROM mcr.microsoft.com/dotnet/aspnet:8.0
25
26 WORKDIR /App
27
28 COPY --from=build-env /App/out .
29
30 ENTRYPOINT ["dotnet", "DotNet.Docker.dll"]
```

Figura 1 – Dockerfile dotnet
Fonte: Elaborado pelo autor (2024)

Em “Pipelines”, edite a pipeline existente e o passo que envia a imagem para o ACR deve estar nos stages de deploy. É possível inseri-lo em todos os stages relacionados a deploy ou somente em um específico. Neste caso, vamos adicionar ao stage Dev.

Antes da inserção do passo que envia a imagem, o stage fica da seguinte forma (figura 2):



```
43 .....task: DotNetCoreCLI@2
44 .....displayName: 'Restore project dependencies'
45 .....inputs:
46 .....command: 'restore'
47 .....projects: '**/*.csproj'
48
49 .....Settings
50 .....task: DotNetCoreCLI@2
51 .....displayName: 'Build the project - $(buildConfiguration)'
52 .....inputs:
53 .....command: 'build'
54 .....arguments: '--no-restore --configuration $(buildConfiguration)'
55 .....projects: '**/*.csproj'
56 .....Settings
57 .....task: DotNetCoreCLI@2
58 .....displayName: 'Publish the project - $(buildConfiguration)'
59 .....inputs:
60 .....command: 'publish'
61 .....projects: '**/*.csproj'
62 .....publishWebProjects: false
63 .....arguments: '--no-build --configuration $(buildConfiguration) --output $(Build.ArtifactStagingDirectory)/$(buildConfiguration)'
64 .....zipAfterPublish: true
65 .....publish: '$(Build.ArtifactStagingDirectory)'
66 .....artifact: drop
67 .....
```

Figura 2 – Stage Build
Fonte: Elaborado pelo autor (2024)

Logo abaixo da linha 66 da figura 2, depois de artifact devemos inserir o campo – task e apontar a task usada para o step em questão. Neste caso temos Docker@2, responsável por utilizar o módulo de Docker para executar comandos relacionados a ele. Em seguida, podemos adicionar um displayName para identificar na execução da pipeline; vamos nomeá-lo como “Build and Push an image to container registry”. Uma descrição objetiva ajuda quem está lendo o arquivo a identificar o que de fato a task fará.

Depois temos os inputs, comandos e informações necessárias para que a imagem vá para a cloud correta, o repositório correto e com o nome de imagem correto. O primeiro deve ser o input “command”, o comando usado da task que será

empregado. Nesse caso, faremos o build e o push da imagem; então devemos setar buildAndPush.

Dentre as opções possíveis, podemos usar uma da lista a seguir:

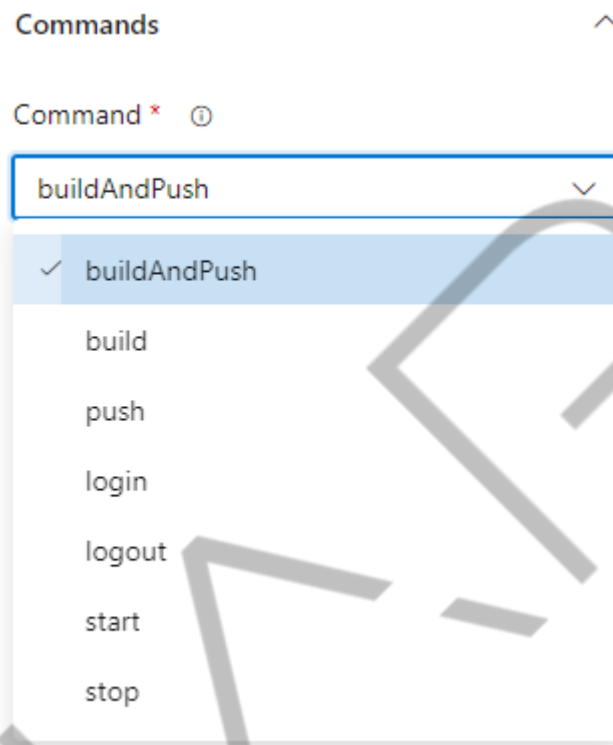


Figura 3 – Commands
Fonte: Elaborado pelo autor (2024)

O próximo comando deve ser o “repository”, apontando o nome de repositório usado em que a imagem ficará salva.

Como esse campo pode mudar com o tempo, não devemos inserir o valor diretamente, podemos usar uma variável para isso. Dessa forma, uma variável pode ser apontada com \$(nome-variável), que terá o valor somente do nome do repositório. Nesta aula, vamos usar o nome de fiap-azure, mas fique à vontade para utilizar o valor que faça mais sentido para você.

O comando seguinte deve ser o “dockerfile”, campo que indicará onde o arquivo Dockerfile se encontra. Por padrão o arquivo costuma ficar na raiz do Repos, então podemos definir “****/Dockerfile**” apontando para raiz + nome do arquivo.

O penúltimo comando está relacionado ao container registry criado, neste campo o valor deve ser coletado por meio de conexão com o Registry. Para isso,

clique em “Project Settings” no canto esquerdo inferior e em seguida em “Service Connections”. Em seguida, no canto superior direito, há a opção “New Service Connection”; selecione a opção “Docker Registry”.

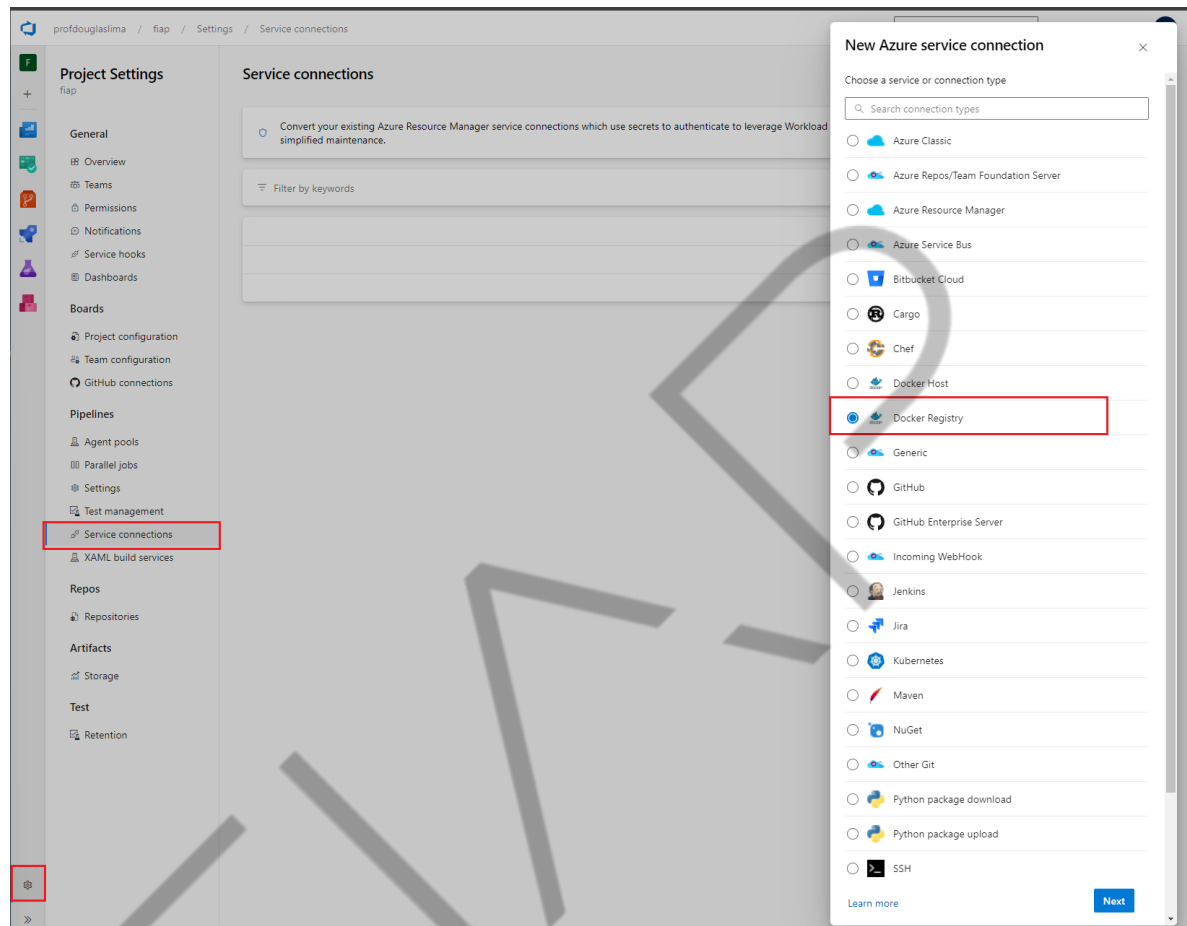


Figura 4 – Service connections
Fonte: Elaborado pelo autor (2024)

Avançando, é necessário selecionar a opção “Azure Container Registry” em “Registry Type”.

Em “Authentication Type”, podemos escolher “Service Principal” e em seguida selecionar nossa subscription/conta da Azure. Isso abrirá uma tela de login da Microsoft para logar na conta e gerar o token de acesso.

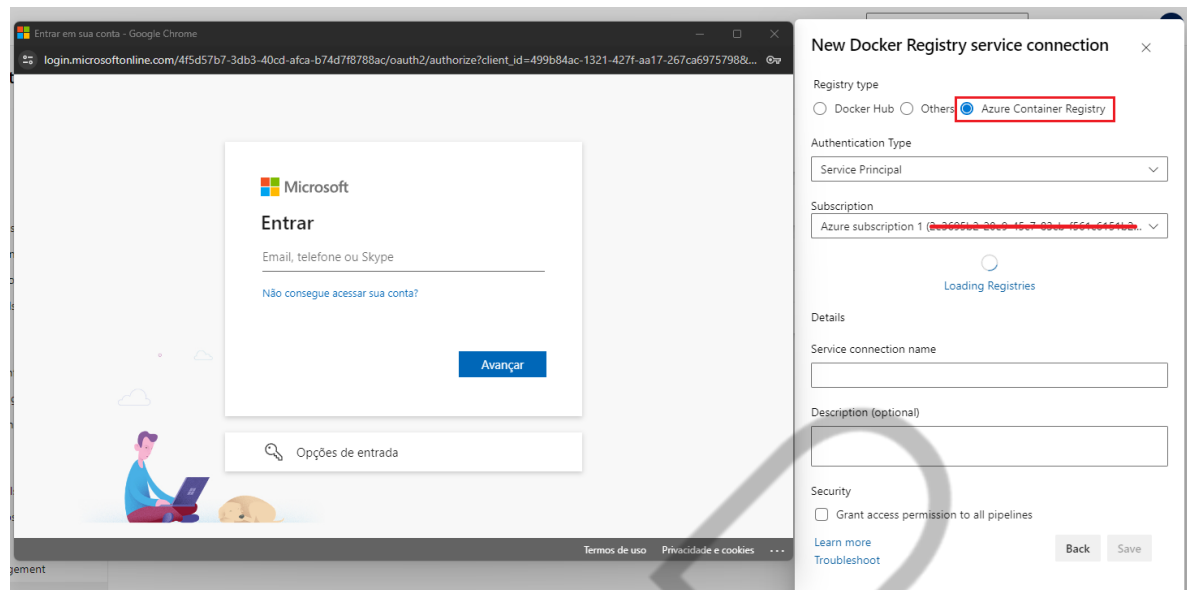


Figura 5 – New Docker Registry
Fonte: Elaborado pelo autor (2024)

Ao logar na conta, será necessário selecionar o ACR criado. Nesta aula, criamos o fiapregistry. Em “Service Connection Name” devemos nomear esse “Service Connections”; ele será o identificador em nossas conexões e esse nome será utilizado no apontamento da pipeline a seguir.

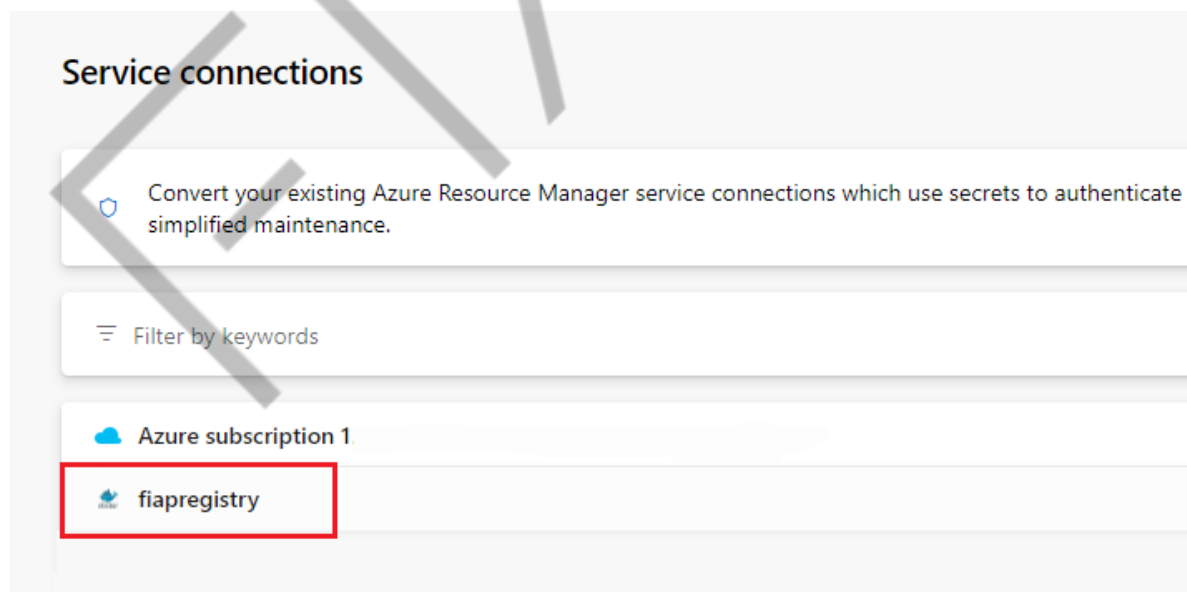


Figura 6 – Service connections list
Fonte: Elaborado pelo autor (2024)

Por último temos o comando tags, que será responsável por inserir a tag correta que será usada no push. Ela organiza e identifica o repositório e registry usado.

Voltando para pipelines, vamos editar a existente, navegar até a task de Docker e inserir o Docker Registry Service Connection criado anteriormente. Basta ir até a task, selecionar “Settings” e o “Service Connections”, neste caso fiapregistry.

Se preferir, você pode pular a parte de Settings e escrever na pipeline o nome do service connections, isso já basta. A task de build deve ficar dessa maneira:

```
Settings
- task: Docker@2
  displayName: Build and push an image to container registry
  inputs:
    containerRegistry: 'fiapregistry'
    repository: '$(imageRepository)'
    command: 'buildAndPush'
    Dockerfile: '**/Dockerfile'
    tags: '$(Build.BuildId)'
```

Figura 7 – Pipeline
Fonte: Elaborado pelo autor (2024)

E a única variável setada foi a imageRepository, com o nome do repository. O valor deve ser parecido com o da figura 8.

```
variables:
  buildConfiguration: 'Release'
  releaseBranchName: 'release'
  imageRepository: 'fiap-azure'
```

Figura 8 – Pipeline vars
Fonte: Elaborado pelo autor (2024)

Com a pipeline pronta, podemos salvá-la e executá-la para ver a saída.

The screenshot displays the Azure DevOps pipeline interface. On the left, the 'Jobs in run #20240124.9' for the 'fiap' pipeline are listed. The 'Build job' is expanded, showing steps like 'Initialize job', 'Checkout fiap@azure-p...', 'Use .NET SDK 6.x', 'Run npm install', 'Compile Sass assets', 'Write build info', 'Restore project depend...', 'Build the project - Rele...', 'Publish the project - Re...', 'PublishPipelineArtifact', 'Build and push an ...', 'Post-job: Checkout fiap...', and 'Finalize Job'. The 'Build and push an ...' step is highlighted with a red box.

The main panel shows the terminal output for the 'Build and push an image to container registry' step. The output includes a table of repository images and a list of image history entries.

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
***fiap-azure	51	c087b249b001	5 seconds ago	220MB
fiapregistry/samples/dotnet-teste	latest	86ba9e709dc	3 hours ago	220MB
dotnet-teste	latest	86ba9e709dc	3 hours ago	220MB
***dotnet-teste	latest	86ba9e709dc	3 hours ago	220MB
mcr.microsoft.com/dotnet/aspnet	8.0	c390b050530f	13 days ago	217MB
fiapregistry/samples/hello-world	latest	fce289e99eb9	5 years ago	1.84kB
mcr.microsoft.com/mcr/hello-world	latest	fce289e99eb9	5 years ago	1.84kB

The output also includes a list of image history entries, showing the creation of the image 'fiap-azure:51' and its subsequent pushes to the container registry.

Figura 9 – Pipeline push ACR
Fonte: Elaborado pelo autor (2024)

O push da imagem foi feito corretamente. Voltando no Azure Container Registry/Repositórios, podemos notar um segundo repositório chamado fiap-azure com a imagem enviada pelo Pipelines.

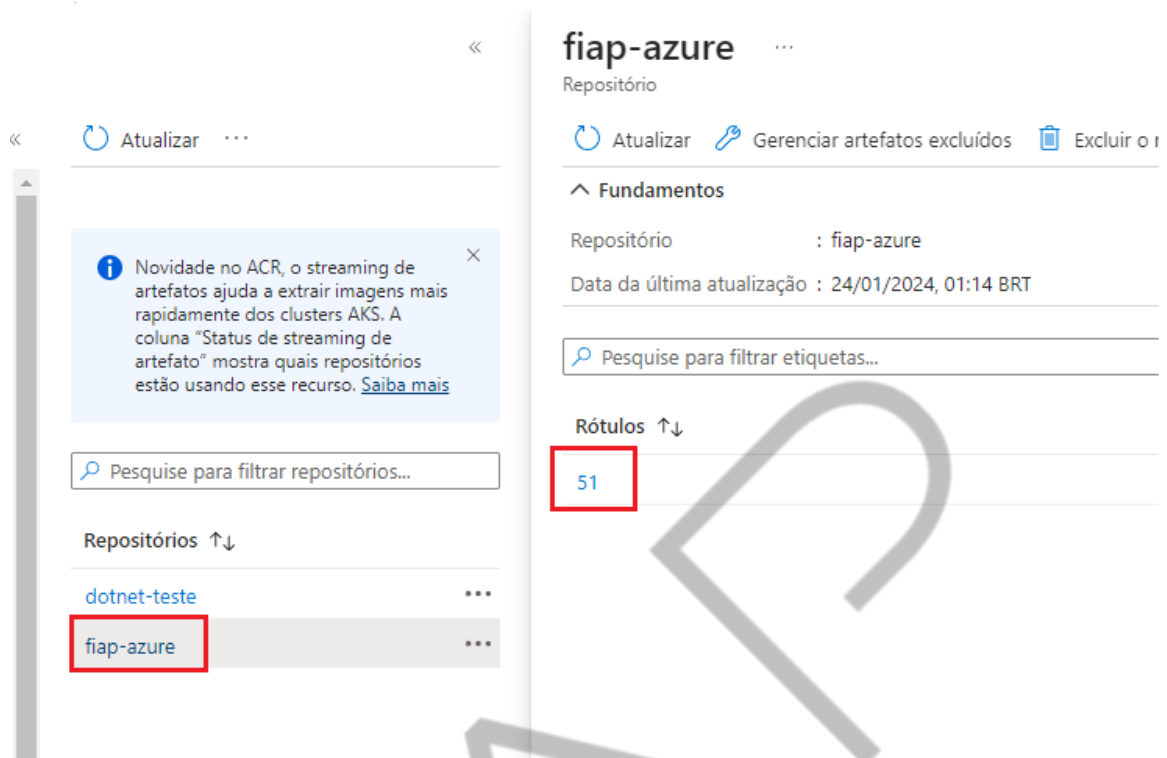


Figura 10 – ACR Repo
Fonte: Elaborado pelo autor (2024)

A partir de agora, com todo novo push o build será executado e uma nova imagem será enviada para o ACR.

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, você viu como gerar uma imagem docker por meio do Azure Pipelines e enviá-la para o Azure Container Registry de forma automática, sem a necessidade de executar o processo de criação de imagem e push para o repositório criado no ACR.

EXEMPLO

REFERÊNCIAS

MICROSOFT BUILD. **Build and push Docker images to Azure Container Registry using Docker templates**. 2023. Disponível em: <<https://learn.microsoft.com/en-us/azure/devops/pipelines/ecosystems/containers/acr-template?view=azure-devops>>. Acesso em: 29 abr. 2024.

MICROSOFT BUILD. **Definir recursos no YAML**. 2023. Disponível em: <<https://learn.microsoft.com/pt-br/azure/devops/pipelines/process/resources?view=azure-devops&tabs=schema#troubleshooting-authorization-for-a-yaml-pipeline>>. Acesso em: 29 abr. 2024.

MICROSOFT BUILD. **Manage service connections**. 2022. Disponível em: <<https://learn.microsoft.com/en-us/azure/devops/pipelines/library/service-endpoints?view=azure-devops&tabs=yaml>>. Acesso em: 29 abr. 2024.

PALAVRAS-CHAVE

Palavras-chave: Docker. ACR. Azure Pipelines.

EMSE

POS TECH